Logbook Kegiatan — 27 April 2026

Penelitian	Sistem Kendali Drone Kamikaze Berbasis Deteksi Objek Warna dalam Simulasi HITL
Tim	Musa El Hanafi & Muhammad Ihsan Fahriansyah
Lokasi	Lab Komputer SMA Swasta Alfa Centauri, Kota Bandung
Hari/Tanggal	Senin, 27 April 2026

Kegiatan hari ini berfokus pada pengembangan modul **drone-seeker** — komponen computer vision yang bertanggung jawab mendeteksi dan melacak target berwarna untuk sistem kendali drone kamikaze. Kegiatan dibagi dalam lima bagian: **Bagian 1** pengenalan OpenCV dan konsep dasar ruang warna; **Bagian 2** implementasi aplikasi dasar (`imshow` dan konversi BGR→HSV); **Bagian 3** implementasi pipeline deteksi dan pelacakan objek berdasarkan warna; **Bagian 4** implementasi aplikasi kalibrasi warna interaktif menggunakan trackbar; **Bagian 5** implementasi histogram ROI interaktif dan pelacakan warna dengan algoritma CamShift.

Arsitektur Modul drone-seeker

Modul `drone-seeker` bertugas memproses frame kamera secara real-time, mendeteksi target berwarna, dan mengirimkan perintah kendali ke Pixhawk via MAVLink. OpenCV menjadi inti pipeline pemrosesan visual.

```
Kamera / Video Feed
       │
       ▼
  Frame Capture (VideoCapture)
       │
       ▼
  Konversi BGR → HSV
       │
       ▼
  Masking berdasarkan rentang Hue/Sat/Val
       │
       ▼
  Morphological Filtering (erode + dilate)
       │
       ▼
  Contour Detection → Centroid Kalkulasi
       │
       ▼
  Error Kalkulasi (ex, ey dari pusat frame)
       │
       ▼
  MAVLink Command (velocity setpoint / attitude)
       │
       ▼
      Pixhawk (HITL via PPP)
```

Bagian 1 — Mengenal OpenCV

1. Pengenalan OpenCV

Kegiatan:
Mempelajari OpenCV sebagai library computer vision utama yang digunakan dalam modul `drone-seeker` untuk deteksi dan pelacakan target berwarna.

Apa itu OpenCV:

OpenCV (Open Source Computer Vision Library) adalah library open-source untuk pemrosesan gambar dan video secara real-time. Library ini mendukung Python, C++, dan Java, serta berjalan di Linux, macOS, dan Windows.

Aspek	Keterangan
Nama lengkap	Open Source Computer Vision Library
Versi yang digunakan	OpenCV 4.x
Bahasa	Python 3 (binding `cv2`)
Lisensi	Apache 2.0
Website	https://opencv.org

Konsep Dasar yang Dipelajari:

Konsep	Penjelasan
Image sebagai array NumPy	Gambar direpresentasikan sebagai `ndarray` dengan shape `(H, W, 3)` untuk BGR atau `(H, W)` untuk grayscale
Ruang warna BGR	OpenCV menggunakan urutan Blue-Green-Red (bukan RGB) secara default
Ruang warna HSV	Hue-Saturation-Value — lebih robust untuk segmentasi warna di bawah perubahan pencahayaan
Masking	Binary image hasil threshold yang digunakan untuk mengisolasi piksel target
Contour	Kurva yang menghubungkan piksel dengan intensitas sama — digunakan untuk menemukan batas objek

Mengapa HSV lebih baik dari BGR untuk deteksi warna:

Kondisi	BGR	HSV
Cahaya redup	Semua channel turun — warna berubah	Hue tetap stabil, hanya Value turun
Bayangan	Warna menjadi gelap dan tidak konsisten	Hue tidak berubah, Saturation sedikit turun
Sorotan lampu	Channel meledak ke nilai tinggi	Value naik, Hue tetap bisa di-mask
Implementasi	Sulit — 3 threshold per warna	Mudah — 1 range Hue + toleransi Sat/Val

Instalasi:

```
pip install opencv-python numpy
```

Hasil: Konsep dasar OpenCV, representasi gambar sebagai array NumPy, dan keunggulan ruang warna HSV untuk deteksi warna dipahami.

Bagian 2 — Aplikasi Dasar: Imshow dan BGR→HSV

---

2. Aplikasi Imshow

Kegiatan:
Membangun aplikasi kamera secara bertahap: pertama menampilkan feed kamera, lalu menambahkan fitur rekam video menggunakan ffmpeg.

Tahap 1 — Menampilkan Gambar dari Kamera

Kode:

```
import cv2

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

if not cap.isOpened():
    print("Error: kamera tidak ditemukan")
    exit()

print(f"Resolusi:
{int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))}x{int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))}")
print("Tekan 'q' untuk keluar")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: gagal membaca frame dari kamera")
        break

    cv2.imshow("Kamera", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Penjelasan fungsi utama:

Fungsi	Keterangan
`cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)`	Buka kamera dengan backend AVFoundation (macOS native) — menghindari warning backend lain
`cap.isOpened()`	Cek apakah kamera berhasil dibuka
`cap.read()`	Ambil satu frame; kembalikan `(ret, frame)` — `ret=False` jika gagal
`cv2.imshow(name, img)`	Tampilkan frame dalam named window
`cv2.waitKey(1)`	Tunggu input keyboard 1 ms; loop tetap berjalan real-time
`cap.release()`	Lepas handle kamera
`cv2.destroyAllWindows()`	Tutup semua window OpenCV

Hasil: Feed kamera tampil di window OpenCV secara real-time. Tekan `q` untuk keluar.

Tahap 2 — Menambahkan Fungsi Rekam Video (`app_imshow.py`)

Rekaman menggunakan **`cv2.VideoWriter`** native OpenCV dengan codec MJPG — tidak membutuhkan dependensi eksternal.

Cara kerja: Tiap frame BGR ditulis langsung ke `cv2.VideoWriter` → dikompres dengan codec MJPG dan disimpan ke `.avi`.

Kode (`app_imshow.py`):

```
import cv2
import time

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

if not cap.isOpened():
    print("Error: kamera tidak ditemukan")
    exit()

width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))  // 2
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)) // 2
fps    = cap.get(cv2.CAP_PROP_FPS) or 30.0

print(f"Resolusi: {width}x{height} @ {fps:.0f} FPS")
print("Tekan 'r' untuk mulai/stop rekam | 's' untuk screenshot | 'q' untuk keluar")

writer    = None
recording = False

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: gagal membaca frame dari kamera")
        break

    frame = cv2.resize(frame, (width, height))

    if recording and writer is not None:
        writer.write(frame)

    display = frame.copy()

    if recording:
        cv2.circle(display, (20, 20), 8, (0, 0, 255), -1)
        cv2.putText(display, "REC", (35, 27),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    cv2.imshow("Kamera", display)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('s'):
        filename = f"screenshot_{int(time.time())}.png"
        cv2.imwrite(filename, frame)
        print(f"Screenshot disimpan: {filename}")
    elif key == ord('r'):
        if not recording:
            filename = f"rekaman_{int(time.time())}.avi"
            writer = cv2.VideoWriter(
                filename,
                cv2.VideoWriter_fourcc(*"MJPG"),
                fps,
                (width, height),
            )
            recording = True
            print(f"Rekaman dimulai: {filename}")
        else:
            recording = False
            writer.release()
```

```
            writer = None
            print("Rekaman dihentikan dan disimpan.")

if recording and writer is not None:
    writer.release()

cap.release()
cv2.destroyAllWindows()
```

Fitur tambahan dibanding Tahap 1:

Fitur	Keterangan
Resize 50% di awal	`width//2`, `height//2` — frame dikecilkan sebelum proses apapun; rekaman juga resolusi 50%
`display = frame.copy()`	Gambar indikator REC di `display`, bukan `frame` — rekaman tidak terkena overlay
Rekam via `cv2.VideoWriter`	Frame ditulis langsung ke VideoWriter; dikompres dengan codec MJPG ke `.avi`

Penjelasan tambahan (fitur rekam):

Fungsi / Parameter	Keterangan
`cv2.VideoWriter(filename, fourcc, fps, (w, h))`	Buat objek writer — tentukan file output, codec, FPS, dan resolusi
`cv2.VideoWriter_fourcc(*"MJPG")`	Codec Motion JPEG — tersedia di semua instalasi OpenCV tanpa dependensi eksternal
`writer.write(frame)`	Tulis satu frame BGR ke file
`writer.release()`	Finalisasi dan tutup file rekaman

Kontrol keyboard:

Tombol	Fungsi
`r`	Mulai rekam → tekan lagi untuk stop dan simpan
`s`	Simpan screenshot `screenshot_<timestamp>.png`
`q`	Keluar; rekaman aktif otomatis di-finalisasi sebelum keluar

Cara menjalankan:

```
python app_imshow.py
```

Hasil: Feed kamera tampil real-time dengan indikator ● `REC` saat merekam. File rekaman tersimpan sebagai `.avi` MJPG tanpa dependensi eksternal.

---

3. Aplikasi Konversi BGR → HSV

Kegiatan:
Membuat aplikasi untuk memvisualisasikan perbedaan antara ruang warna BGR dan HSV, sekaligus memahami cara kerja `cv2.cvtColor`.

Kode (`app_bgr2hsv.py`):

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

if not cap.isOpened():
```

```
    print("Error: kamera tidak ditemukan")
    exit()

print("Tekan 'q' untuk keluar")

def half(img):
    return cv2.resize(img, None, fx=0.5, fy=0.5)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: gagal membaca frame dari kamera")
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    h, s, v = cv2.split(hsv)

    vis_top = np.hstack([frame, hsv])
    vis_bot = np.hstack([
        cv2.cvtColor(h, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(s, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(v, cv2.COLOR_GRAY2BGR),
    ])

    # Label kolom di vis_top
    fw = frame.shape[1]
    cv2.putText(vis_top, "BGR", (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
    cv2.putText(vis_top, "HSV", (fw + 10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    # Label kolom di vis_bot
    for img, labels in [(vis_bot, ["H", "S", "V"])]:
        for i, label in enumerate(labels):
            cv2.putText(img, label, (i * fw + 10, 25),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    # Nilai piksel di titik tengah frame
    cy, cx = frame.shape[0] // 2, frame.shape[1] // 2
    bgr_val = frame[cy, cx]
    hsv_val = hsv[cy, cx]
    info = f"Pusat — BGR:{bgr_val.tolist()}  HSV:{hsv_val.tolist()}"
    cv2.putText(vis_top, info, (10, vis_top.shape[0] - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)

    cv2.imshow("BGR | HSV", half(vis_top))
    cv2.imshow("H | S | V", half(vis_bot))

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Rentang nilai HSV di OpenCV:

Channel	Rentang OpenCV	Keterangan
H (Hue)	0 – 179	OpenCV menggunakan setengah dari 360°
S (Saturation)	0 – 255	0 = putih/abu-abu, 255 = warna penuh

Channel	Rentang OpenCV	Keterangan
V (Value)	0 – 255	0 = hitam, 255 = terang penuh

> **Penting:** Hue di OpenCV = Hue_sebenarnya / 2. Merah = 0–10 dan 170–179 (melingkar).

Referensi Hue untuk warna umum:

Warna	H (OpenCV)	H (360°)
Merah	0–10 dan 170–179	0° dan 350–360°
Oranye	10–25	20–50°
Kuning	25–35	50–70°
Hijau	35–85	70–170°
Cyan	85–100	170–200°
Biru	100–130	200–260°
Ungu/Magenta	130–170	260–340°

Hasil: Aplikasi BGR→HSV berhasil menampilkan visualisasi semua channel secara bersamaan. Pemahaman tentang rentang nilai HSV di OpenCV tercapai.

Bagian 3 — Aplikasi Deteksi dan Pelacakan Objek Berdasarkan Warna

---

4. Pipeline Deteksi Objek Berwarna

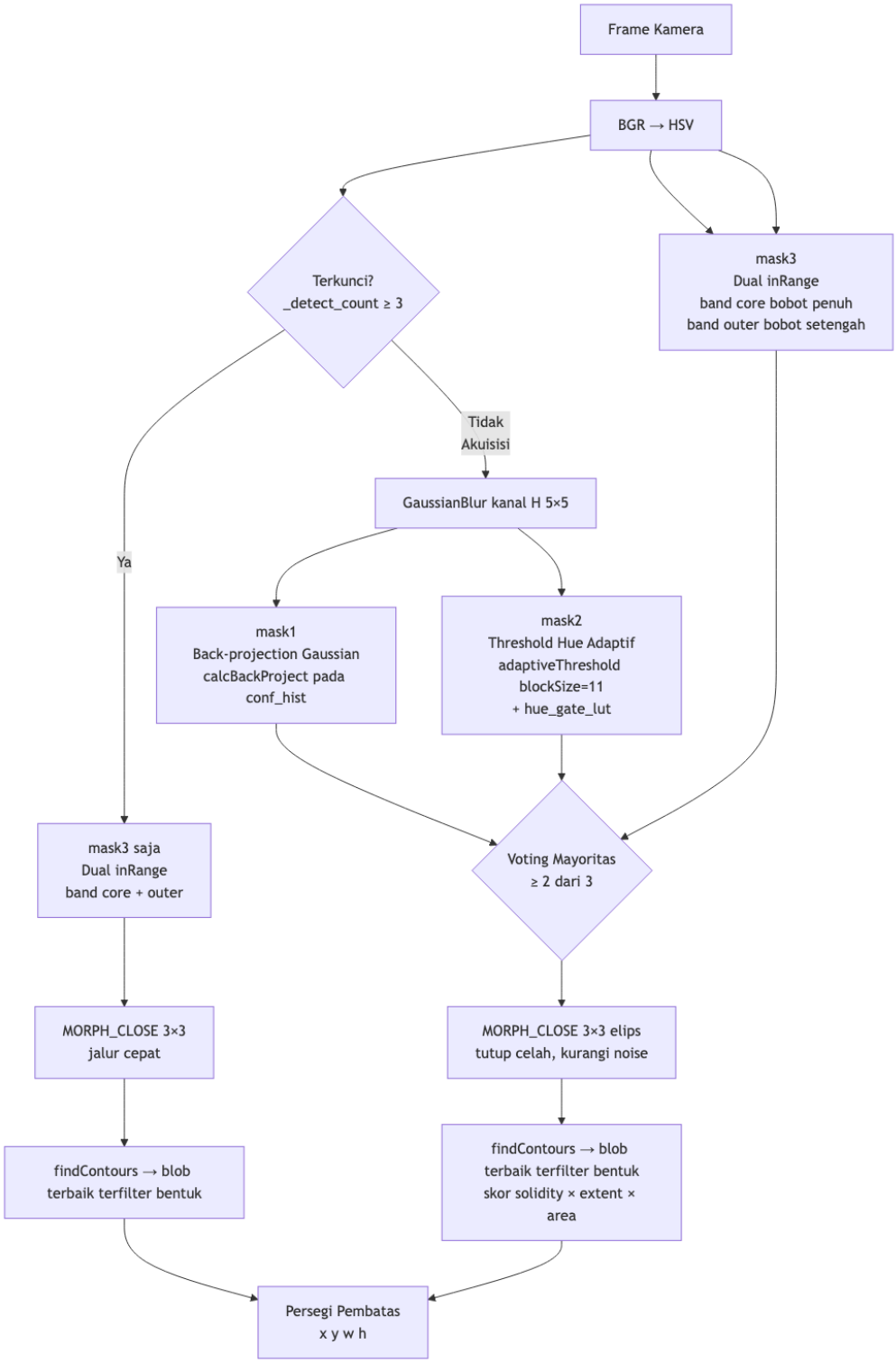
Kegiatan:
Membangun pipeline lengkap untuk mendeteksi objek berdasarkan warna dalam video real-time, menampilkan bounding box dan centroid objek yang terdeteksi.

Kode (`app_color_tracker.py`):

```
import cv2
import numpy as np

# --- Konfigurasi warna target (contoh: oranye) ---
# Rentang HSV untuk warna oranye
HSV_LOWER = np.array([10, 120, 100])
HSV_UPPER = np.array([25, 255, 255])

# Ukuran minimum contour yang dianggap valid (piksel persegi)
MIN_AREA = 500

def get_mask(hsv_frame, lower, upper):
    """Buat binary mask untuk rentang HSV yang diberikan."""
    mask = cv2.inRange(hsv_frame, lower, upper)
    # Morphological opening: hilangkan noise kecil
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.erode(mask, kernel, iterations=2)
    mask = cv2.dilate(mask, kernel, iterations=2)
    return mask

def find_largest_contour(mask):
    """Temukan contour terbesar dari mask."""
    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
```

```
    )
    if not contours:
        return None
    return max(contours, key=cv2.contourArea)

def draw_target(frame, contour, frame_cx, frame_cy):
    """Gambar bounding box, centroid, dan error ke frame."""
    area = cv2.contourArea(contour)
    if area < MIN_AREA:
        return None, None

    # Bounding box
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Centroid via moments
    M = cv2.moments(contour)
    if M["m00"] == 0:
        return None, None
    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])

    # Gambar centroid
    cv2.circle(frame, (cx, cy), 6, (0, 0, 255), -1)

    # Error dari pusat frame
    dx = cx - frame_cx
    dy = cy - frame_cy
    cv2.arrowedLine(frame, (frame_cx, frame_cy), (cx, cy), (255, 0, 0), 2)

    # Label area dan koordinat
    label = f"Area={int(area)} | dx={dx} dy={dy}"
    cv2.putText(frame, label, (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    return dx, dy

# --- Main loop ---
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    h_frame, w_frame = frame.shape[:2]
    frame_cx, frame_cy = w_frame // 2, h_frame // 2

    # Gambar crosshair tengah frame
    cv2.drawMarker(frame, (frame_cx, frame_cy),
                   (200, 200, 200), cv2.MARKER_CROSS, 20, 1)

    # Pipeline deteksi
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = get_mask(hsv, HSV_LOWER, HSV_UPPER)
    contour = find_largest_contour(mask)

    dx, dy = None, None
    if contour is not None:
        dx, dy = draw_target(frame, contour, frame_cx, frame_cy)
```

```
    # Status di pojok kiri atas
    status = f"TARGET DETECTED | dx={dx} dy={dy}" if dx is not None else "NO TARGET"
    color = (0, 255, 0) if dx is not None else (0, 0, 255)
    cv2.putText(frame, status, (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

    # Tampilkan frame utama dan mask
    cv2.imshow("Tracker", frame)
    cv2.imshow("Mask", mask)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Penjelasan langkah pipeline:

Langkah	Fungsi OpenCV	Output
Konversi warna	`cv2.cvtColor(frame, BGR2HSV)`	Frame dalam ruang HSV
Threshold warna	`cv2.inRange(hsv, lower, upper)`	Binary mask (0 atau 255)
Erode	`cv2.erode(mask, kernel, 2)`	Hilangkan noise kecil (piksel terisolasi)
Dilate	`cv2.dilate(mask, kernel, 2)`	Pulihkan ukuran objek setelah erode
Temukan contour	`cv2.findContours(mask, ...)`	List semua kontur pada mask
Hitung centroid	`cv2.moments(contour)`	Koordinat pusat massa objek (cx, cy)

Langkah	Fungsi OpenCV	Output
Bounding box	`cv2.boundingRect(contour)`	Kotak pembatas (x, y, w, h)

Mengapa erode → dilate (bukan sebaliknya):
- **Erode** terlebih dahulu: menghilangkan noise kecil dan piksel tepi yang tidak diinginkan
- **Dilate** setelahnya: mengembalikan ukuran objek yang terkikis oleh erode
- Urutan ini disebut **morphological opening** — efektif untuk menghilangkan noise halus tanpa menghilangkan objek utama

Hasil: Pipeline deteksi warna berjalan real-time. Centroid target berhasil dihitung dan error posisi (dx, dy) dari pusat frame berhasil diukur.

---

5. Menangani Warna Merah (Hue Melingkar)

Kegiatan:
Implementasi khusus untuk deteksi warna merah yang memiliki Hue melingkar di rentang 0 dan 179 pada skala OpenCV.

Kode tambahan untuk masking merah:

```
def get_red_mask(hsv_frame):
    """
    Warna merah memiliki Hue di dua ujung rentang (0–10 dan 170–179).
    Perlu dua mask yang digabungkan dengan bitwise OR.
    """
    lower_red1 = np.array([0,   120, 100])
    upper_red1 = np.array([10,  255, 255])
    lower_red2 = np.array([170, 120, 100])
    upper_red2 = np.array([179, 255, 255])

    mask1 = cv2.inRange(hsv_frame, lower_red1, upper_red1)
    mask2 = cv2.inRange(hsv_frame, lower_red2, upper_red2)

    # Gabungkan kedua mask
    mask = cv2.bitwise_or(mask1, mask2)
    return mask
```

Hasil: Deteksi warna merah berhasil dengan menggabungkan dua rentang Hue.

Bagian 4 — Aplikasi Kalibrasi Warna

---

6. Aplikasi Kalibrasi Warna Interaktif

Kegiatan:
Membuat aplikasi kalibrasi interaktif menggunakan trackbar OpenCV untuk menentukan rentang HSV yang tepat bagi target berwarna pada kondisi pencahayaan tertentu. Nilai HSV hasil kalibrasi akan digunakan sebagai konstanta dalam `app_color_tracker.py`.

Kode (`app_color_calibrator.py`):

```
import cv2
import numpy as np
import json
import os

WINDOW_CTRL  = "Kalibrasi Warna — Kontrol HSV"
```

```
WINDOW_ORIG  = "Original"
WINDOW_MASK  = "Mask"
WINDOW_RESULT = "Hasil Masking"
OUTPUT_FILE  = "color_config.json"

def nothing(x):
    pass

def create_trackbars():
    cv2.namedWindow(WINDOW_CTRL, cv2.WINDOW_NORMAL)
    cv2.resizeWindow(WINDOW_CTRL, 400, 300)

    # Trackbar Lower HSV
    cv2.createTrackbar("H Min", WINDOW_CTRL,   0, 179, nothing)
    cv2.createTrackbar("S Min", WINDOW_CTRL,  50, 255, nothing)
    cv2.createTrackbar("V Min", WINDOW_CTRL,  50, 255, nothing)
    # Trackbar Upper HSV
    cv2.createTrackbar("H Max", WINDOW_CTRL, 179, 179, nothing)
    cv2.createTrackbar("S Max", WINDOW_CTRL, 255, 255, nothing)
    cv2.createTrackbar("V Max", WINDOW_CTRL, 255, 255, nothing)

def get_trackbar_values():
    h_min = cv2.getTrackbarPos("H Min", WINDOW_CTRL)
    s_min = cv2.getTrackbarPos("S Min", WINDOW_CTRL)
    v_min = cv2.getTrackbarPos("V Min", WINDOW_CTRL)
    h_max = cv2.getTrackbarPos("H Max", WINDOW_CTRL)
    s_max = cv2.getTrackbarPos("S Max", WINDOW_CTRL)
    v_max = cv2.getTrackbarPos("V Max", WINDOW_CTRL)
    return (
        np.array([h_min, s_min, v_min]),
        np.array([h_max, s_max, v_max])
    )

def save_config(lower, upper, label="target"):
    config = {
        label: {
            "hsv_lower": lower.tolist(),
            "hsv_upper": upper.tolist()
        }
    }
    # Merge dengan config yang sudah ada
    if os.path.exists(OUTPUT_FILE):
        with open(OUTPUT_FILE, "r") as f:
            existing = json.load(f)
        existing.update(config)
        config = existing

    with open(OUTPUT_FILE, "w") as f:
        json.dump(config, f, indent=2)
    print(f"Konfigurasi disimpan ke {OUTPUT_FILE}: lower={lower.tolist()} upper={upper.tolist()}")

def load_config(label="target"):
    if not os.path.exists(OUTPUT_FILE):
        return None, None
    with open(OUTPUT_FILE, "r") as f:
        config = json.load(f)
    if label not in config:
        return None, None
    lower = np.array(config[label]["hsv_lower"])
    upper = np.array(config[label]["hsv_upper"])
```

```
    return lower, upper

create_trackbars()

# Load konfigurasi sebelumnya jika ada
prev_lower, prev_upper = load_config()
if prev_lower is not None:
    cv2.setTrackbarPos("H Min", WINDOW_CTRL, int(prev_lower[0]))
    cv2.setTrackbarPos("S Min", WINDOW_CTRL, int(prev_lower[1]))
    cv2.setTrackbarPos("V Min", WINDOW_CTRL, int(prev_lower[2]))
    cv2.setTrackbarPos("H Max", WINDOW_CTRL, int(prev_upper[0]))
    cv2.setTrackbarPos("S Max", WINDOW_CTRL, int(prev_upper[1]))
    cv2.setTrackbarPos("V Max", WINDOW_CTRL, int(prev_upper[2]))
    print("Konfigurasi sebelumnya dimuat.")

cap = cv2.VideoCapture(0)

print("=" * 50)
print("Kalibrasi Warna HSV")
print("  S = simpan konfigurasi ke color_config.json")
print("  Q = keluar tanpa menyimpan")
print("=" * 50)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lower, upper = get_trackbar_values()

    # Buat mask
    mask = cv2.inRange(hsv, lower, upper)

    # Morphological cleaning
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask_clean = cv2.erode(mask, kernel, iterations=1)
    mask_clean = cv2.dilate(mask_clean, kernel, iterations=1)

    # Terapkan mask ke frame asli
    result = cv2.bitwise_and(frame, frame, mask=mask_clean)

    # Tampilkan nilai HSV di piksel tengah sebagai panduan
    cy, cx = frame.shape[0] // 2, frame.shape[1] // 2
    hsv_center = hsv[cy, cx]
    info = f"Pusat frame HSV: H={hsv_center[0]} S={hsv_center[1]} V={hsv_center[2]}"
    cv2.putText(frame, info, (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1)

    # Gambar crosshair di tengah
    cv2.drawMarker(frame, (cx, cy), (0, 255, 255), cv2.MARKER_CROSS, 20, 1)

    # Tampilkan rentang aktif
    range_info = f"Lower HSV: {lower.tolist()}  |  Upper HSV: {upper.tolist()}"
    cv2.putText(result, range_info, (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)

    cv2.imshow(WINDOW_ORIG,   frame)
    cv2.imshow(WINDOW_MASK,   mask_clean)
    cv2.imshow(WINDOW_RESULT, result)
```

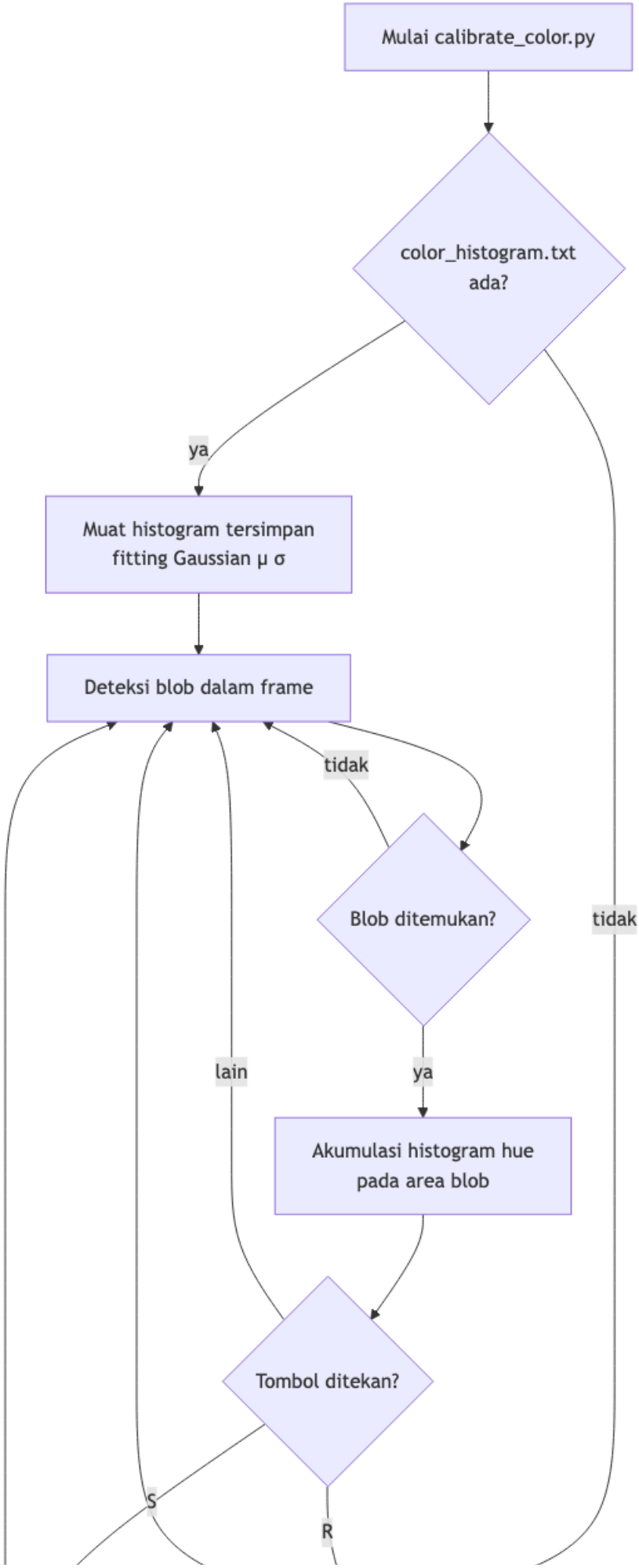
```
    key = cv2.waitKey(1) & 0xFF
    if key == ord('s'):
        save_config(lower, upper)
    elif key == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

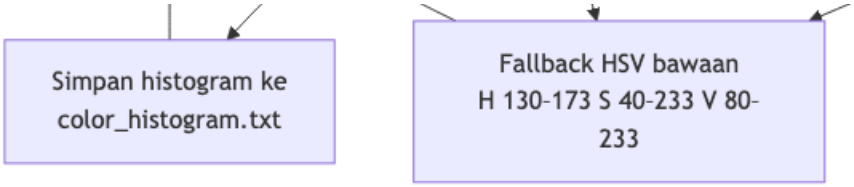
Cara menggunakan hasil kalibrasi di tracker:

```
import json
import numpy as np

def load_color_config(label="target", path="color_config.json"):
    with open(path) as f:
        cfg = json.load(f)
    return (
        np.array(cfg[label]["hsv_lower"]),
        np.array(cfg[label]["hsv_upper"])
    )

# Di app_color_tracker.py — ganti konstanta hardcode dengan:
HSV_LOWER, HSV_UPPER = load_color_config("target")
```

Contoh output `color_config.json`:

```
{
  "target": {
    "hsv_lower": [10, 120, 100],
    "hsv_upper": [25, 255, 255]
  }
}
```


Mulai calibrate_color.py
color_histogram.txt ada?
ya
tidak
Muat histogram tersimpan fitting Gaussian μ σ
Deteksi blob dalam frame
tidak
Blob ditemukan?
lain
ya
Akumulasi histogram hue pada area blob
Tombol ditekan?
S
R



Prosedur kalibrasi yang direkomendasikan:

Langkah	Tindakan
1	Tempatkan target di depan kamera pada kondisi pencahayaan aktual
2	Arahkan crosshair tengah frame ke objek target
3	Baca nilai H, S, V yang ditampilkan di layar
4	Set H Min/Max ±10 dari nilai H target
5	Turunkan S Min hingga mask mencakup seluruh objek, tapi tidak background
6	Turunkan V Min jika area bayangan objek tidak terdeteksi
7	Verifikasi: mask hanya menutupi target, background bersih
8	Tekan **S** untuk menyimpan ke `color_config.json`

Troubleshooting kalibrasi:

Masalah	Kemungkinan Penyebab	Solusi
Mask terlalu banyak noise	S Min atau V Min terlalu rendah	Naikkan S Min ke ≥100
Objek tidak terdeteksi	Rentang Hue terlalu sempit	Perlebar H Min dan H Max masing-masing ±5
Mask hilang di area bayangan	V Min terlalu tinggi	Turunkan V Min ke 50–70
Background ikut termasking	Warna objek mirip background	Pertimbangkan pencahayaan yang lebih terkontrol

Hasil: Aplikasi kalibrasi warna interaktif berhasil dibuat. Rentang HSV target dapat ditentukan secara visual dan disimpan ke `color_config.json` untuk digunakan oleh modul tracker.

Bagian 5 — Histogram dan CamShift Tracking

---

7. Aplikasi Histogram ROI Interaktif

Kegiatan:
Membuat aplikasi untuk memilih region of interest (ROI) pada frame kamera menggunakan mouse, lalu menampilkan histogram Hue dan histogram BGR dari region tersebut secara real-time, lengkap dengan visualisasi mean dan standar deviasi.

Kode (`app_histogram.py`):

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

if not cap.isOpened():
    print("Error: kamera tidak ditemukan")
    exit()

print("Klik dan drag untuk pilih region | 'r' reset | 'q' keluar")
```

```
roi_start = None
roi_end   = None
drawing   = False

def on_mouse(event, x, y, flags, param):
    global roi_start, roi_end, drawing
    if event == cv2.EVENT_LBUTTONDOWN:
        roi_start = (x, y)
        roi_end   = (x, y)
        drawing   = True
    elif event == cv2.EVENT_MOUSEMOVE and drawing:
        roi_end = (x, y)
    elif event == cv2.EVENT_LBUTTONUP:
        roi_end = (x, y)
        drawing = False

cv2.namedWindow("Kamera")
cv2.setMouseCallback("Kamera", on_mouse)

HIST_W = 540
HIST_H = 200
BGR_CHANNELS = [(0, (255, 80,  80),  "B"),
                (1, (80,  200, 80),  "G"),
                (2, (80,  80,  255), "R")]
HSV_CHANNELS = [(1, (80,  200, 200), "S"),
                (2, (200, 200, 200), "V")]

def draw_hsv_histogram(roi_hsv):
    panels = []

    # Panel H — per-bin hue color, range 0–179
    img   = np.zeros((HIST_H, HIST_W, 3), dtype=np.uint8)
    hist  = cv2.calcHist([roi_hsv], [0], None, [180], [0, 180])
    cv2.normalize(hist, hist, 0, HIST_H - 40, cv2.NORM_MINMAX)
    bin_w = HIST_W // 180
    for i in range(180):
        h         = int(hist[i])
        color_bgr = cv2.cvtColor(np.uint8([[[i, 255, 200]]]), cv2.COLOR_HSV2BGR)[0][0].tolist()
        cv2.rectangle(img, (i * bin_w, HIST_H - 40 - h), ((i + 1) * bin_w, HIST_H - 40), color_bgr,
-1)

    mean, std = cv2.meanStdDev(roi_hsv[:, :, 0])
    m, s = mean[0, 0], std[0, 0]
    mx   = int(m * HIST_W / 180)
    sx0  = int(max(0,   m - s) * HIST_W / 180)
    sx1  = int(min(179, m + s) * HIST_W / 180)

    cv2.line(img, (mx, 0), (mx, HIST_H - 40), (255, 255, 255), 2)
    lx = mx + 3 if mx < HIST_W - 40 else mx - 35
    cv2.putText(img, f"{m:.1f}", (lx, 14), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)
    cv2.putText(img, f"{m-s:.1f}", (max(0, sx0 - 2), 30), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (160, 160,
160), 1)
    cv2.putText(img, f"{m+s:.1f}", (min(HIST_W - 35, sx1 + 2), 30), cv2.FONT_HERSHEY_SIMPLEX, 0.4,
(160, 160, 160), 1)
    for i in range(0, 181, 30):
        x = i * bin_w
        cv2.line(img, (x, HIST_H - 45), (x, HIST_H - 40), (180, 180, 180), 1)
        cv2.putText(img, str(i), (x + 2, HIST_H - 27), cv2.FONT_HERSHEY_SIMPLEX, 0.35, (200, 200,
200), 1)
    cv2.putText(img, f"H  mean={m:.1f}  std={s:.1f}", (10, HIST_H - 10), cv2.FONT_HERSHEY_SIMPLEX,
```

```
0.5, (255, 255, 255), 1)
    panels.append(img)

    # Panel S dan V — struktur sama dengan BGR
    for ch, color, label in HSV_CHANNELS:
        img  = np.zeros((HIST_H, HIST_W, 3), dtype=np.uint8)
        hist = cv2.calcHist([roi_hsv], [ch], None, [256], [0, 256])
        cv2.normalize(hist, hist, 0, HIST_H - 40, cv2.NORM_MINMAX)
        for i in range(256):
            h  = int(hist[i])
            x0 = i * HIST_W // 256
            x1 = (i + 1) * HIST_W // 256
            cv2.rectangle(img, (x0, HIST_H - 40 - h), (x1, HIST_H - 40), [c // 3 for c in color],
-1)

        mean, std = cv2.meanStdDev(roi_hsv[:, :, ch])
        m, s = mean[0, 0], std[0, 0]
        mx   = int(m * HIST_W / 256)
        sx0  = int(max(0,   m - s) * HIST_W / 256)
        sx1  = int(min(255, m + s) * HIST_W / 256)

        cv2.line(img, (mx, 0), (mx, HIST_H - 40), color, 2)
        lx = mx + 3 if mx < HIST_W - 40 else mx - 35
        cv2.putText(img, f"{m:.1f}", (lx, 14), cv2.FONT_HERSHEY_SIMPLEX, 0.45, color, 1)
        cv2.putText(img, f"{m-s:.1f}", (max(0, sx0 - 2), 30), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (160,
160, 160), 1)
        cv2.putText(img, f"{m+s:.1f}", (min(HIST_W - 35, sx1 + 2), 30), cv2.FONT_HERSHEY_SIMPLEX,
0.4, (160, 160, 160), 1)
        for i in range(0, 257, 64):
            x = i * HIST_W // 256
            cv2.line(img, (x, HIST_H - 45), (x, HIST_H - 40), (140, 140, 140), 1)
            cv2.putText(img, str(i), (x + 2, HIST_H - 27), cv2.FONT_HERSHEY_SIMPLEX, 0.35, (160,
160, 160), 1)
        cv2.putText(img, f"{label}  mean={m:.1f}  std={s:.1f}", (10, HIST_H - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
        panels.append(img)

    return np.vstack(panels)

def draw_bgr_histogram(roi_bgr):
    panel_h = HIST_H
    panels  = []

    for ch, color, label in BGR_CHANNELS:
        img = np.zeros((panel_h, HIST_W, 3), dtype=np.uint8)

        hist = cv2.calcHist([roi_bgr], [ch], None, [256], [0, 256])
        cv2.normalize(hist, hist, 0, panel_h - 40, cv2.NORM_MINMAX)

        bin_w = HIST_W // 256
        for i in range(256):
            h = int(hist[i])
            x0 = i * HIST_W // 256
            x1 = (i + 1) * HIST_W // 256
            cv2.rectangle(img, (x0, panel_h - 40 - h), (x1, panel_h - 40),
                          [c // 3 for c in color], -1)

        mean, std = cv2.meanStdDev(roi_bgr[:, :, ch])
        m, s = mean[0, 0], std[0, 0]
```

```
        mx  = int(m * HIST_W / 256)
        sx0 = int(max(0,   m - s) * HIST_W / 256)
        sx1 = int(min(255, m + s) * HIST_W / 256)

        cv2.line(img, (mx, 0), (mx, panel_h - 40), color, 2)

        lx = mx + 3 if mx < HIST_W - 40 else mx - 35
        cv2.putText(img, f"{m:.1f}", (lx, 14),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.45, color, 1)
        cv2.putText(img, f"{m-s:.1f}", (max(0, sx0 - 2), 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (160, 160, 160), 1)
        cv2.putText(img, f"{m+s:.1f}", (min(HIST_W - 35, sx1 + 2), 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (160, 160, 160), 1)
        for i in range(0, 257, 64):
            x = i * HIST_W // 256
            cv2.line(img, (x, panel_h - 45), (x, panel_h - 40), (140, 140, 140), 1)
            cv2.putText(img, str(i), (x + 2, panel_h - 27),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.35, (160, 160, 160), 1)
        cv2.putText(img, f"{label}  mean={m:.1f}  std={s:.1f}",
                    (10, panel_h - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

        panels.append(img)

    return np.vstack(panels)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: gagal membaca frame dari kamera")
        break

    hsv     = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    display = cv2.resize(frame, None, fx=0.5, fy=0.5)

    if roi_start and roi_end:
        cv2.rectangle(display, roi_start, roi_end, (0, 255, 0), 2)

        # Koordinat pada frame full-res (display = 50%)
        x1 = min(roi_start[0], roi_end[0]) * 2
        y1 = min(roi_start[1], roi_end[1]) * 2
        x2 = max(roi_start[0], roi_end[0]) * 2
        y2 = max(roi_start[1], roi_end[1]) * 2

        if x2 > x1 and y2 > y1:
            roi_bgr = frame[y1:y2, x1:x2]
            roi_hsv = hsv[y1:y2, x1:x2]
            cv2.imshow("Histogram HSV", draw_hsv_histogram(roi_hsv))
            cv2.imshow("Histogram BGR", draw_bgr_histogram(roi_bgr))

    cv2.imshow("Kamera", display)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('r'):
        roi_start = roi_end = None

cap.release()
cv2.destroyAllWindows()
```

Penjelasan elemen visualisasi histogram:

Elemen	Keterangan
Bar histogram	Frekuensi kemunculan tiap nilai channel di dalam ROI
Garis vertikal	Mean — nilai rata-rata channel di region yang dipilih
Angka di atas garis	Nilai mean secara numerik
Angka kiri/kanan garis	Nilai mean−std dan mean+std

Cara koordinat ROI dikonversi:

Display di-resize 50%, sehingga koordinat mouse di display perlu dikalikan 2 untuk mendapatkan koordinat di frame full-resolution yang digunakan `calcHist`.

Kontrol:

Tombol	Fungsi
Klik + drag	Pilih region
`r`	Reset ROI
`q`	Keluar

Hasil: Histogram HSV (panel H, S, V) dan BGR (panel B, G, R) dari region yang dipilih tampil real-time beserta mean dan std dev tiap channel.

---

8. Aplikasi CamShift Color Tracking

Kegiatan:
Mengimplementasikan algoritma CamShift (Continuously Adaptive Mean-Shift) untuk melacak objek berdasarkan warna secara real-time. Target dipilih dengan klik-drag; histogram Hue target digunakan sebagai referensi tracking di setiap frame.

Cara kerja CamShift:

```
Seleksi ROI → hitung histogram Hue target
       │
       ▼
Setiap frame:
  calcBackProject(hsv, roi_hist)
       │  → peta probabilitas: putih = piksel cocok dengan warna target
       ▼
  CamShift(backproj, track_window, criteria)
       │  → geser dan sesuaikan ukuran kotak ke area probabilitas tertinggi
       ▼
  Gambar RotatedRect + centroid di frame
```

Kode (`app_camshift.py`):

```
import cv2
import numpy as np
import time

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

if not cap.isOpened():
    print("Error: kamera tidak ditemukan")
    exit()

print("Klik dan drag untuk pilih target | 'v' rekam | 's' screenshot | 'r' reset | 'q' keluar")
```

```
roi_start    = None
roi_end      = None
drawing      = False
needs_init   = False   # flag: hitung histogram setelah LBUTTONUP
tracking     = False
track_window = None
roi_hist     = None
writer       = None
recording    = False

def on_mouse(event, x, y, flags, param):
    global roi_start, roi_end, drawing, tracking, needs_init
    if event == cv2.EVENT_LBUTTONDOWN:
        roi_start  = (x, y)
        roi_end    = (x, y)
        drawing    = True
        tracking   = False
        needs_init = False
    elif event == cv2.EVENT_MOUSEMOVE and drawing:
        roi_end = (x, y)
    elif event == cv2.EVENT_LBUTTONUP:
        roi_end    = (x, y)
        drawing    = False
        needs_init = True   # sinyal ke main loop untuk init sekali

cv2.namedWindow("CamShift Tracker")
cv2.setMouseCallback("CamShift Tracker", on_mouse)

criteria = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)
HSV_LOW  = np.array([0,  60,  32])
HSV_HIGH = np.array([180, 255, 255])

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: gagal membaca frame dari kamera")
        break

    frame   = cv2.resize(frame, None, fx=0.5, fy=0.5)
    hsv     = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    display = frame.copy()

    # Gambar kotak seleksi saat drag
    if drawing and roi_start and roi_end:
        cv2.rectangle(display, roi_start, roi_end, (0, 255, 0), 2)

    # Inisialisasi tracking — hanya terpicu sekali per seleksi
    if needs_init:
        needs_init = False
        x1 = min(roi_start[0], roi_end[0])
        y1 = min(roi_start[1], roi_end[1])
        x2 = max(roi_start[0], roi_end[0])
        y2 = max(roi_start[1], roi_end[1])

        if x2 > x1 and y2 > y1:
            roi_hsv  = hsv[y1:y2, x1:x2]
            mask     = cv2.inRange(roi_hsv, HSV_LOW, HSV_HIGH)
            roi_hist = cv2.calcHist([roi_hsv], [0], mask, [180], [0, 180])
            cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)
```

```
            track_window = (x1, y1, x2 - x1, y2 - y1)
            tracking = True
        # skip CamShift frame ini — biarkan backproj stabil dulu
        cv2.imshow("CamShift Tracker", display)
        cv2.waitKey(1)
        continue

    # CamShift tracking
    if tracking and roi_hist is not None:
        fh, fw   = frame.shape[:2]
        mask     = cv2.inRange(hsv, HSV_LOW, HSV_HIGH)
        backproj = cv2.calcBackProject([hsv], [0], roi_hist, [0, 180], 1)
        backproj &= mask

        # Clamp track_window ke batas frame sebelum CamShift
        x, y, w, h = track_window
        x = int(max(0, min(x, fw - 2)))
        y = int(max(0, min(y, fh - 2)))
        w = int(max(2, min(w, fw - x)))
        h = int(max(2, min(h, fh - y)))
        track_window = (x, y, w, h)

        ret_cs, track_window = cv2.CamShift(backproj, track_window, criteria)

        pts = np.intp(cv2.boxPoints(ret_cs))
        cv2.polylines(display, [pts], True, (0, 255, 0), 2)

        cx, cy = int(ret_cs[0][0]), int(ret_cs[0][1])
        cv2.circle(display, (cx, cy), 5, (0, 0, 255), -1)

        tw, th = int(ret_cs[1][0]), int(ret_cs[1][1])
        cv2.putText(display, f"cx={cx} cy={cy}  {tw}x{th}px",
                    (10, display.shape[0] - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 1)

        cv2.imshow("Back Projection", backproj)

    if recording and writer is not None:
        writer.write(display)
        cv2.circle(display, (20, 20), 8, (0, 0, 255), -1)
        cv2.putText(display, "REC", (35, 27),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    cv2.imshow("CamShift Tracker", display)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('v'):
        if not recording:
            h, w = display.shape[:2]
            filename = f"rekaman_{int(time.time())}.avi"
            writer = cv2.VideoWriter(
                filename,
                cv2.VideoWriter_fourcc(*"MJPG"),
                cap.get(cv2.CAP_PROP_FPS) or 30.0,
                (w, h),
            )
            recording = True
            print(f"Rekaman dimulai: {filename}")
```

```
        else:
            recording = False
            writer.release()
            writer = None
            print("Rekaman dihentikan dan disimpan.")
    elif key == ord('s'):
        filename = f"screenshot_{int(time.time())}.png"
        cv2.imwrite(filename, display)
        print(f"Screenshot disimpan: {filename}")
    elif key == ord('r'):
        roi_start    = roi_end = None
        drawing      = False
        needs_init   = False
        tracking     = False
        track_window = None
        roi_hist     = None
        cv2.destroyWindow("Back Projection")

if recording and writer is not None:
    writer.release()

cap.release()
cv2.destroyAllWindows()
```

Penjelasan fungsi kunci:

Fungsi / Konsep	Keterangan
`cv2.calcBackProject`	Mengganti tiap piksel dengan probabilitas warnanya cocok dengan `roi_hist`; output = grayscale probability map
`cv2.CamShift`	Jalankan Mean-Shift berulang (maks 10 iterasi) lalu sesuaikan ukuran window; return `RotatedRect` dan `track_window` baru
`cv2.boxPoints(ret_cs)`	Konversi `RotatedRect` ke 4 titik sudut untuk digambar dengan `polylines`
`needs_init` flag	Inisialisasi histogram hanya terpicu sekali saat `LBUTTONUP`; frame init di-`continue` agar CamShift tidak jalan di frame yang sama
Clamp `track_window`	Koordinat dan dimensi dipaksa masuk batas frame sebelum setiap panggilan CamShift untuk mencegah hang
`HSV_LOW / HSV_HIGH`	Mask saturasi ≥ 60 dan value ≥ 32 — menyaring piksel abu-abu/hitam yang tidak representatif

Kontrol:

Tombol	Fungsi
Klik + drag	Pilih target warna
v	Mulai/stop rekam video ke `.avi` MJPG
s	Simpan screenshot `screenshot_<timestamp>.png`
r	Reset tracking, pilih target baru
q	Keluar; rekaman aktif otomatis di-finalisasi

Hasil: CamShift berhasil melacak objek berdasarkan warna secara real-time. Kotak rotasi hijau mengikuti target di setiap frame; window Back Projection menampilkan peta probabilitas warna. Rekaman video dan screenshot dapat disimpan langsung dari aplikasi.

Desain Model UAV di X-Plane Plane Maker

Selain pengembangan modul drone-seeker, pada hari ini juga dilakukan pemodelan tiga varian UAV — **Satria Nano**, **Satria Talon**, dan **Satria Delta** — di **X-Plane Plane Maker** sebagai dasar simulasi HITL. Plane Maker digunakan untuk mendefinisikan geometri sayap, airfoil, titik CG, dan parameter propulsi secara iteratif sehingga menghasilkan model aerodinamis yang realistis sebelum diuji terbang.

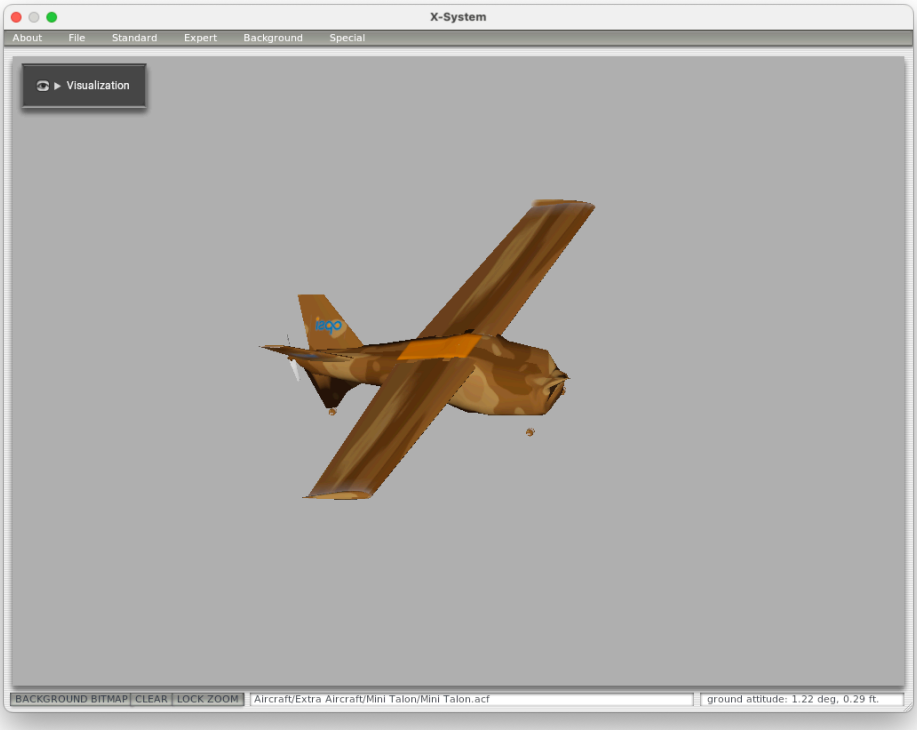
Satria Nano

UAV fixed-wing mini dengan sayap tapered NACA 2412 dan ekor V-tail. Varian terkecil dalam keluarga Satria, dirancang untuk misi ringan dengan payload minimal dan kemudahan pengoperasian.

Parameter	Nilai
Airfoil	**NACA 2412**
Wingspan	**92.66 cm**
Semi-length	**1.52 ft (46.33 cm)**
Wing area	0.1843 m²
Aspect ratio	4.68
Konfigurasi ekor	V-tail twin fin (45°)
Empty weight	**2.20 lb (0.998 kg)**
MTOW	**3.50 lb (1.588 kg)**
Payload	590 g
CG nominal	**0.63 ft = 5.87 cm dari LE MAC**
Wing loading	84.8 N/m²

Parameter	Nilai
Stall speed	**9.61 m/s (34.6 km/h)**
Cruise speed	**15.73 m/s (56.6 km/h)**
L/D max	**10.76**
Cruise power	38 W
Endurance	**~45 mnt**
Range one-way	**~1 km**
Motor	2212, KV 979–1100 @ 3S
Propeller	APC 9×4.5
ESC	30 A BLHeli_32
Baterai	4S 2200 mAh / 3S 3000 mAh

Satria Talon

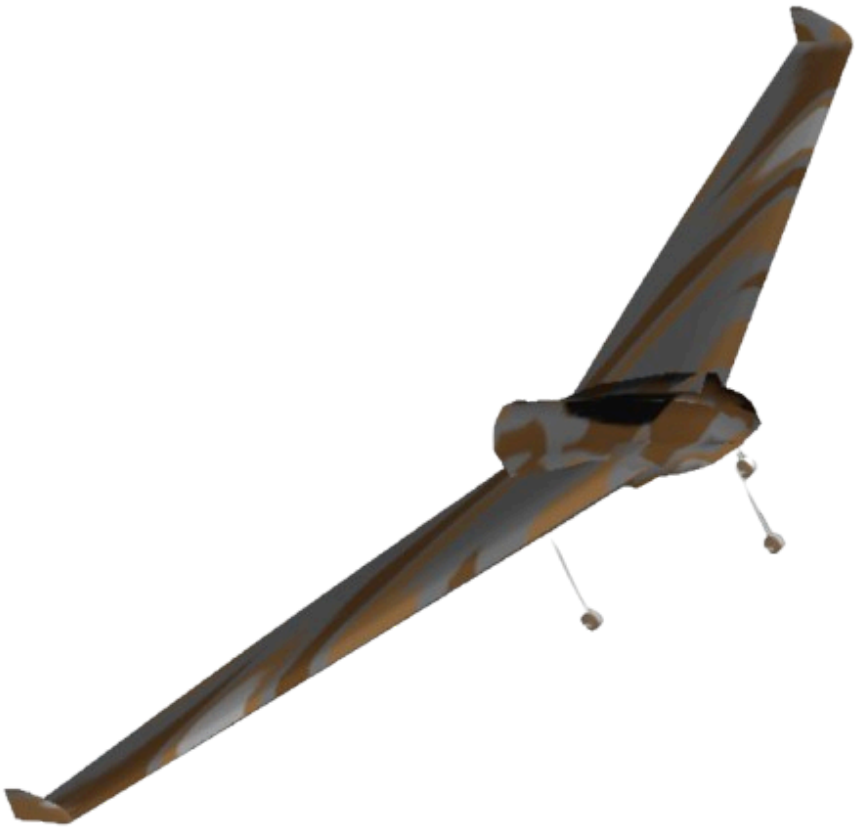


UAV fixed-wing konvensional berukuran medium dengan sayap tapered NACA 2412 dan ekor V-tail. Dirancang untuk misi endurance jarak jauh dengan payload kamera atau muatan misi.

Parameter	Nilai
Airfoil	**NACA 2412**
Wingspan	**~129.8 cm**
Semi-length	**2.13 ft (64.92 cm)**
Wing area	0.2731 m²
Aspect ratio	6.17
Konfigurasi ekor	V-tail twin fin (45°)
Empty weight	**3.00 lb (1.361 kg)**
MTOW	**5.25 lb (2.381 kg)**
Payload (4S 8Ah)	571 g
CG nominal	**0.92 ft = 5.87 cm dari LE MAC**
Wing loading	85.5 N/m²
Stall speed	**9.65 m/s (34.7 km/h)**
Cruise speed	**14.74 m/s (53.1 km/h)**
L/D max	**12.37**
Cruise power	46 W
Endurance	**~131 mnt**
Range one-way	**~116 km**
Radius RTH	**~49–52 km**
Motor	2216, KV 555–600 @ 4S
Propeller	APC 11×5.5E
ESC	40 A BLHeli_32

Parameter	Nilai
Baterai	4S 8000 mAh

Satria Delta



UAV konfigurasi delta wing — sayap menyatu tanpa ekor konvensional. Model ini ditambahkan di X-Plane Plane Maker sebagai varian eksperimental untuk misi kecepatan tinggi dan manuverabilitas. Spesifikasi detail masih dalam tahap finalisasi.

Ringkasan Kegiatan

No	Kegiatan	Status
1	Pengenalan OpenCV — konsep dasar, ruang warna, instalasi	✅ Selesai
2	Aplikasi Imshow — tampilkan gambar statis dan feed kamera	✅ Selesai
3	Aplikasi BGR→HSV — visualisasi konversi ruang warna	✅ Selesai
4	Aplikasi Deteksi Objek — pipeline masking, contour, centroid	✅ Selesai
5	Penanganan warna merah (Hue melingkar)	✅ Selesai
6	Aplikasi Kalibrasi Warna — trackbar interaktif + simpan JSON	✅ Selesai
7	Pemodelan Satria Nano di X-Plane Plane Maker	✅ Selesai
8	Pemodelan Satria Talon di X-Plane Plane Maker	✅ Selesai
9	Pemodelan Satria Delta di X-Plane Plane Maker	✅ Selesai

Logbook dibuat: 27 April 2026 | Penelitian OPSI 2026 — SMA Swasta Alfa Centauri